

AI-Augmented Data Analysis

SQL Server + Copilot / Claude Code / Codex

Delegowanie realnej pracy nad hurtownią danych agentowi AI w sposób procesowy



Training

- plan szkolenia i cele
- obecne obowiązki
- pytania, dyskusje, potrzeby grupy
- aktywne uczestnictwo
- elastyczny program szkolenia
- zachęcam do zadawania pytań



Twój trener

Mateusz Kulesza

- Starszy programista i konsultant
- Lider zespołu i kierownik projektu
- Specjalizuje się w architekturze aplikacji fullstack
- Trener fullstack — od 2016r.



Ty — uczestnik

- Jakie są twoje codzienne obowiązki?
- Z jakimi technologiami pracujesz?
- Czy masz wcześniejsze doświadczenie z AI?
- Jakie są twoje cele na to szkolenie?
- Na jakich obszarach lub tematach chcesz się skupić?

Wprowadzenie do LLMs

Wnioskowanie AI w analitycznym przepływie pracy - błędne założenia i pułapki

Co właściwie delegujemy?

Zanim zaczniemy wdrażanie — czym TO właściwie jest?

Deterministyczne a probabilistyczne

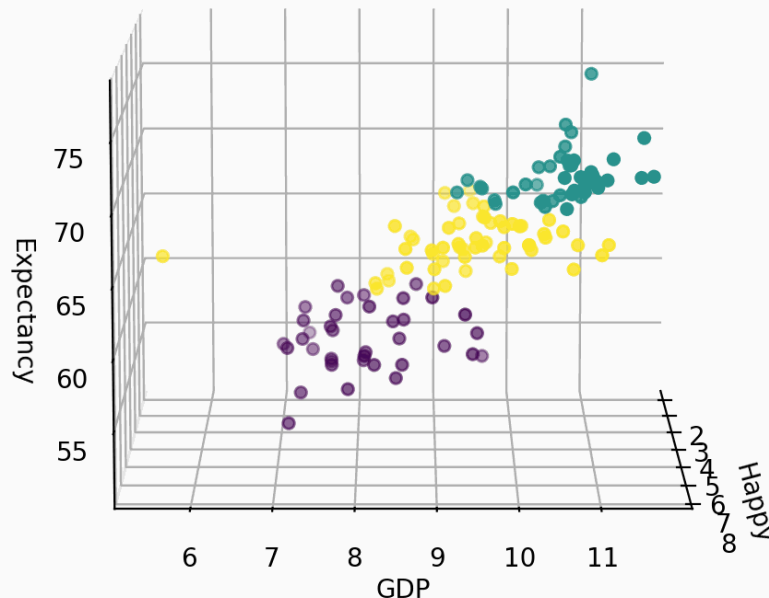
Tradycyjny kod

- To samo wejście → to samo wyjście
- Zawsze przewidywalny
- Weryfikowalny przez inspekcję
- Deterministyczne wykonanie

Generatywne AI

- Wiarygodny wynik, ale niegwarantowany
- Przekonujące ≠ poprawne
- Przewidywanie tokenów, nie obliczanie
- Generacja probabilistyczna

KMeans Clustering of WHR2023 Data

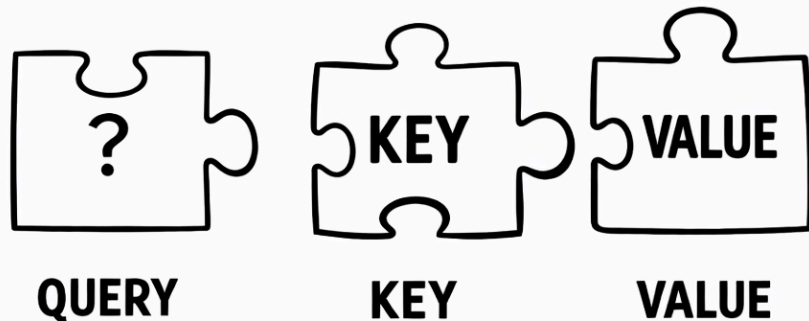


Nie wyszukiwanie, tylko predykcja

AI nie „wyszukuje” tabel wymiarów z ukrytego słownika hurtowni.

- Przewiduje, jak tabele wymiarów *prawdopodobnie* wyglądają
- Na podstawie wzorców widzianych w danych treningowych
- Plus wszystkiego, co podałeś w kontekście
- Żadnego ukrytego algorytmu biznesowego
- Żadnej wewnętrznej bazy wiedzy

TRANSFORMER



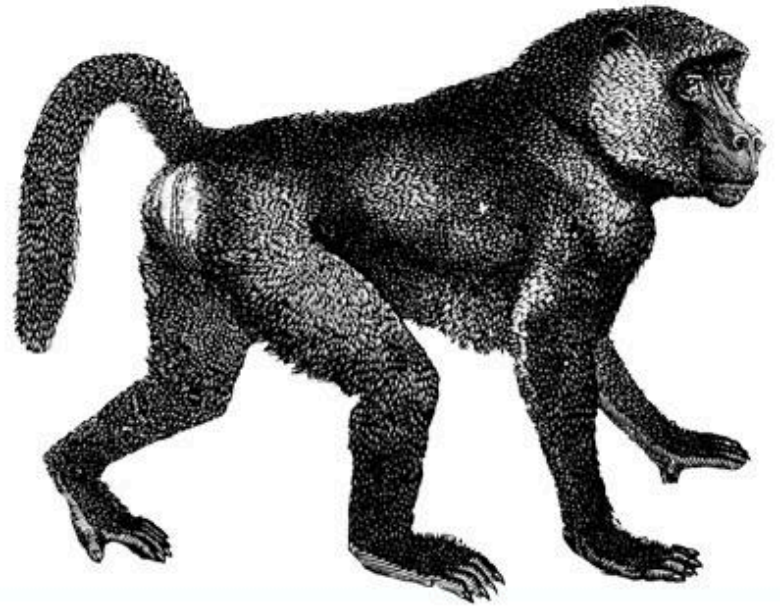
Twarde problemy vs miękkie problemy

Twarde (deterministyczne)

- Joiny i agregacje
- Walidacja danych
- Sprawdzanie ograniczeń
- Obliczenia liczbowe
- → zadanie dla SQL Server / Excel / Python

Miękkie (interpretacja)

- Interpretowanie wymagań
- Tłumaczenie biznes → SQL
- Wyjaśnianie istniejącego kodu
- Rozpoznawanie wzorców
- → zadanie AI



AI based on if /
else statements

The Definitive Guide

Kluczowa myśl

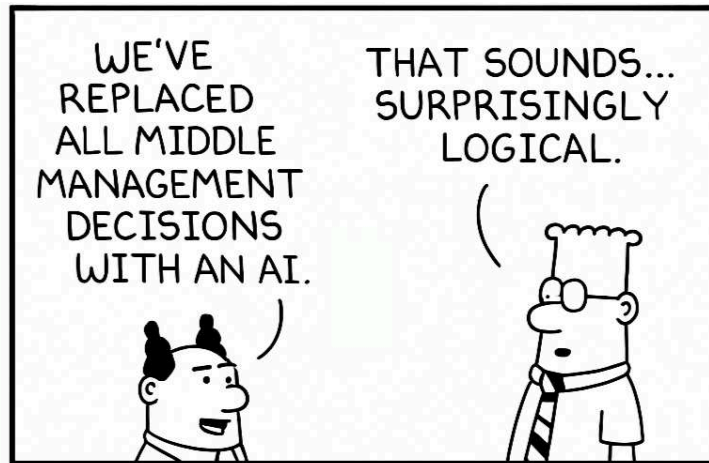
Proszenie AI o konkretny wynik to grzech pierwotny

Człowiek + narzędzia + AI

Najsilniejszy przepływ pracy łączy wszystkie trzy:

- Narzędzia wykonują deterministyczne obliczenia
- Ludzie definiują znaczenie i intencję
- AI łączy je, tłumacząc i generując

Najślabszy przepływ: zastępowanie deterministycznego narzędzia LLM-em.

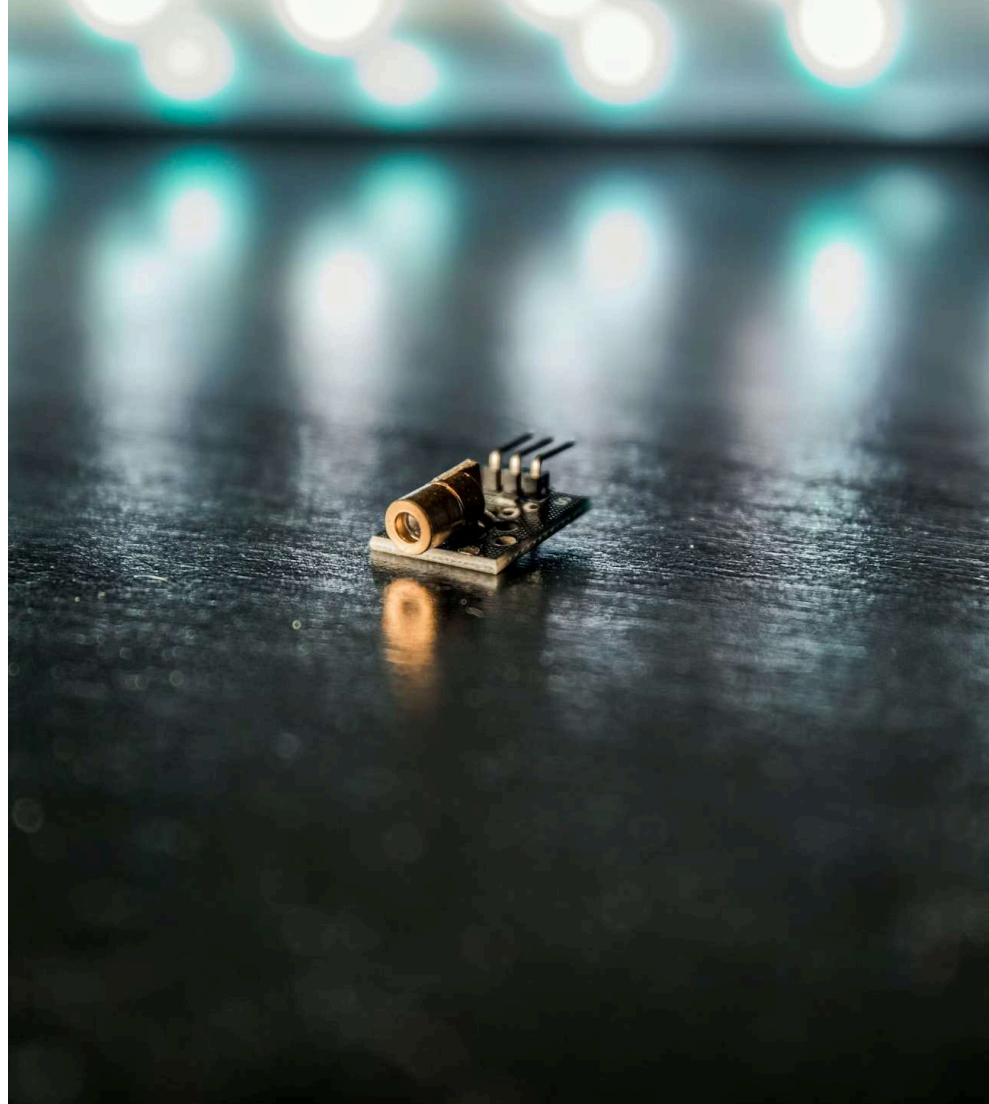


AI jako proces, nie odpowiedź

Każda interakcja z AI to pipeline:

```
input
  → context
    → generation →
      → verification
        → refinement
          → result
```

- Weryfikacja to normalny krok, nie reakcja na awarie
- Ta sama dyscyplina co przy każdej zmianie ETL
- Dev → test → uzgodnienie → deploy



W naszej aplikacji

Cały ten warsztat to jeden długi przykład tego procesu.

- Jedno ewoluujące wymaganie
- Iteracyjna generacja i weryfikacja
- Dopracowanie na podstawie dowodów
- Żaden krok nie opiera się wyłącznie na zaufaniu

Model, API, aplikacja

Trzy warstwy

„Ten asystent jest gorszy od tamtego” to zwykle różnica w kontekście i narzędziach, a nie w modelu.

Tokeny

Tekst wejściowy i wyjściowy mierzy się w tokenach, nie w słowach ani zdaniach.

- Procedura składowana to tysiące tokenów
- Nie „jeden plik” — tylko pozycja budżetowa
- Limity kontekstu to liczby tokenów
- Dłuższe wejście = wyższy koszt

el (LLM), like OpenAI's GPT-3 or called tokenization. Tokenization into smaller units (or tokens) t ess. Tokens can be as small as a even larger in some models. As enization process is largely det the specific tokenizer used dur se of GPT-3 and GPT-4, they use r. BPE is a subword tokenization dynamically create a vocabulary presents common words or word pa the tokenization process might r nt versions of a models (e.g., C

Tury to nie pamięć

Rozmowa czatowa nie ma wrodzonej pamięci.

- Każda tura jest niezależna
- Aplikacja ponownie wysyła wcześniejsze tury jako kontekst
- Tura 12 „zna” turę 3 tylko dlatego, że aplikacja ją ponownie wysłała
- Długie rozmowy sumują koszt tokenów

Kontekst to budżet

Nie wiadro

Pięciowarstwowy stos:

1. System + instrukcje
2. Schemat / DDL
3. Historia rozmowy
4. Wyjście narzędzi
5. Twoje pytanie

Charakterystyka

- Stabilne, możliwe do cache'owania → najtańsze do powtarzania
- Drogie, o dużej wartości → muszą być trafne
- Rośnie po cichu → po 30 turach to głównie szum
- Dowód → najcenniejsze na każdy bajt
- Ostatnie, często najmniejsze → działa na wszystko powyżej

Strefa głupoty

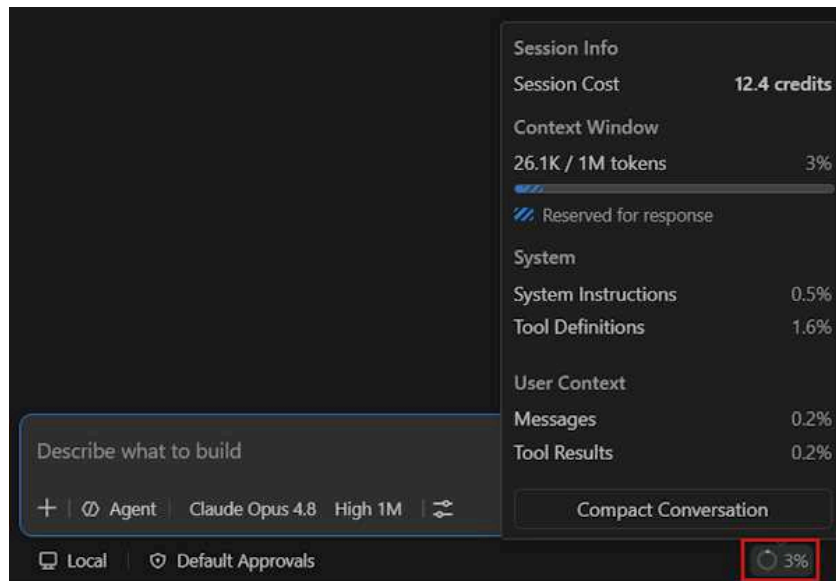
Więcej kontekstu $\neq$ lepsze odpowiedzi

Istotne fakty ukryte w środku kontekstu są odzyskiwane najgorzej, nawet w modelach z długim kontekstem.

U-kształtna krzywa trafności:

wysoka na początku i końcu kontekstu, zapadająca się pośrodku.

Liu et al., „Lost in the Middle”, TACL 2023.



Limit $\neq$ twój schemat

Wiedza wbudowana ma granicę treningu i nigdy nie obejmowała twojego prywatnego schematu.

- Model wie, czym jest Type 2 SCD
- Nie wie, że twój używa kolumny snapshot
- Aktualny DDL, konwencje i reguły muszą przyjść przez kontekst lub narzędzia

Pamięć a uczenie się

Trzy różne pojęcia, często mylone:

- Rozmowa — tury, które aplikacja ponownie wysyła
- Zapisana pamięć — zapisany kontekst, który aplikacja wczytuje ponownie
- Uczenie się — wagi modelu, stałe po treningu

Skorygowanie zapytania nie trenuje niczego na nowo. Aplikacja może je tylko zapisać i ponownie wysłać.

Cache to ekonomia, nie pamięć

Prefix caching obniża koszt i opóźnienie dla stabilnego prefiksu.

- Trafienie w cache nie czyni modelu mądrzejszym
- Pudło nie czyni go głupszym
- Stabilne instrukcje są tanie do powtarzania
- To właśnie zmienność kontekstu kosztuje

W naszej aplikacji

Dlatego warsztat zaczyna się od zaufanej bazy przed jakąkolwiek pracą sterowaną AI.

- Kontekst musi być osadzony w rzeczywistym repozytorium
- Nie założony ani nie zhalucynowany
- Każde zadanie zaczyna się od zweryfikowanych artefaktów

Halucynacja to skutek

Nie kategoria błędu

Generacja przez prawdopodobne dokończenie wymyśla tabele, kolumny i reguły — z pełnym przekonaniem.

Ślad

Zapytanie wygenerowane przez AI odwołuje się do atrybutu wymiaru tygodnia fiskalnego.

- Pewne siebie
- Dobrze sformatowane
- Obok prawdziwego DDL, które tego nie ma

Nie kłamał. Dokańczył wzorzec.

Znane / wnioskowane / brakujące

Wymuś ten podział przed każdą zmianą obejmującą wiele obiektów:

- Znane — odczytane bezpośrednio z dostarczonych plików
- Wnioskowane — wyciągnięte wnioski, ale nie bezpośrednio odczytane
- Brakujące — potrzebne, by nie zgadywać

Najlepszy wynik to czasem pytanie, nie zapytanie.

Strategie weryfikacji

Typ wyjścia determinuje sposób walidacji:

- Wykonaj — czy działa bez błędu?
- Porównaj agregaty — czy sumy się zgadzają?
- Przeczytaj plan — czy używa indeksów poprawnie?
- Uzgodnij — czy liczby przed i po się zgadzają?
- Uruchom testy — czy istniejąca walidacja przechodzi?

Kluczowa myśl

„Zero zwróconych wierszy to dowód”

Zapytanie uzgadniające z pustym zbiorem wyników jest warte więcej niż pewny siebie akapit.

W naszej aplikacji

W niedawnym teście celowo zasianych defektów w zestawie regresyjnym:

- Większość zgłaszała normalny status sukcesu
- Bez żadnego błędu
- Po cichu błędne wyniki
- Tylko zapytania weryfikacyjne je ujawniły

To główne odkrycie warsztatu — i bezpośredni skutek probabilistycznej generacji.

Cztery szczeble

Każdy dodaje możliwości i ryzyko

„AI” to nie jedna rzecz.

Drabina

Cztery odrębne możliwości, często mylone:

- **Model** — token in → token out, bez pamięci
- **Chat** — generuje na podstawie kontekstu rozmowy
- **Asystent** — trwałe instrukcje, pliki, niektóre narzędzia
- **Agent** — planuje i wykonuje wiele działań prowadzących do celu

Narzędzia zmieniają to, co AI może zrobić

Bez narzędzi:

- Rozumuje tylko nad tym, co wkleiłeś
- Zgaduje zawartość plików
- Nie może zweryfikować własnych twierdzeń

Z dostępem do systemu plików / bazy danych / terminala:

- Ogląda rzeczywiste artefakty
- Może obalić własne przypuszczenia
- grep lepszy niż zgadywanie

Kluczowa myśl

Agent to czat, który może się pomylić na głos — a potem to sprawdzić

Autonomia a kontrola

Większa autonomia nie jest automatycznie lepsza.

- Uprawnienia mają znaczenie
- Tam, gdzie to możliwe, dostęp tylko do odczytu
- Bramki akceptacji dla zmian
- Maksymalna automatyzacja $\neq$ najlepszy przepływ pracy

Omówione już na żywo

Ta drabina interakcji jest omawiana ponownie w późniejszym module:

- Zastosowana bezpośrednio do repozytorium hurtowni danych
- Żywa demonstracja każdego szczebla
- Prawdziwe użycie narzędzi, nie abstrakcyjne przykłady

Jeśli czytasz to przed warsztatem: zobaczysz to jeszcze raz, w wyższej rozdzielczości.

Prompt to lekka specyfikacja

Dobre prompty analityka przypominają zgłoszenia robocze, a nie pytania.

Sześć elementów

Każdy skuteczny prompt zawiera:

- Instrukcja — czasownik (analizuj, napisz, przepis, wyjaśnij)
- Rola — perspektywa, nie udawana wiedza
- Kontekst — schematy, próbki, konwencje
- Specyfikacja — format i struktura wyjścia
- Przykłady — jeden prawdziwy plik > trzy akapity
- Kryteria sukcesu — nazwij sędziego (testy, uzgodnienie, walidacja)

Czy zleciłbyś to juniorowi?

Nieprecyzyjne

„Jak dodać kolumnę do tabeli faktów?”

W formie zgłoszenia

- Cel: procedura ładowania tabeli faktów
- Zmiana: dodaj kolumnę audytową
- Propagacja przez: staging → transformacja → merge
- Sukces: kolumna obecna, brak utraty danych, testy przechodzą
- Kontekst: [załączony DDL]

CRISPE jako lista kontrolna

Context · Role · Instruction · Specification · Performance · Examples

- Nie formularz do mechanicznego wypełniania
- Wykrywacz brakujących informacji
- Używaj tego, co istotne, pomijaj resztę

Istnieją też inne frameworki (COSTAR, CARE, TRACE, ERA, GRADE) — wszystkie są wariacjami tego samego motywu.

Omówione już na żywo

To pojawia się ponownie w późniejszym module:

- Zastosowane bezpośrednio do rzeczywistej zmiany
- Budowanie promptu na żywo
- Rzeczywista weryfikacja wyników

Jeśli to jest dla ciebie nowe: poćwiczysz to później.

Jeden doskonały prompt to mit

Złożona analiza używa przepływu pracy, nie jednego promptu.

Artefakty, nie opisy

Dawaj prawdziwą procedurę, DDL i metadane — nigdy tylko własne wspomnienie o nich.

- „Nasza tabela produktów ma około 20 kolumn” ❌
- [attached: CREATE TABLE dbo.DimItem] ✅

Każde zdanie, które piszesz o kodzie, jest stratną kompresją kodu.

Sprawdź przed zmianą

Dla nieznanego SQL lub ETL:

- Najpierw poproś o zależności i przepływ danych
- Najpierw zrozum, co robi obecny kod
- Zidentyfikuj skutki w dół łańcucha
- Dopiero potem zmień

Oddziel eksplorację od wykonania

AI świetnie generuje kandydaty.

- Generuj propozycje
- Oceniaj w testach
- Zmiany produkcyjne nadal przechodzą przez: testy, przegląd, wdrożenie

Eksploracja ≠ produkcja.

Docelowy model mentalny

AI to probabilistyczny komponent wnioskowania i generacji działający na dostarczonym kontekście, wewnątrz kontrolowanego procesu analitycznego.

Każde zadanie sprowadza się do czterech pytań:

1. Jakiego kontekstu potrzebuje?
2. Jakiego wyjścia oczekuję?
3. Jakich narzędzi powinno użyć?
4. Jak zweryfikuję wynik?

Część 1

Analiza, kontekst i pierwsza delegowana zmiana

Zanim przekażemy cokolwiek

Co to WŁAŚCIWIE jest to, co mamy powierzyć tej hurtowni danych?

Cztery pytania

Każde zadanie odpowiada na te pytania, niezależnie od tego, czy je wypowiesz:

1. Jakiego kontekstu potrzebuje?
2. Co ma wytworzyć?
3. Jakich narzędzi może użyć?
4. Jak zweryfikujemy wynik?

Uruchom to dwa razy

Pierwsza próba:

```
MERGE fact_table AS target
USING staging_table AS source
ON target.product_id = source.product_id
   AND target.date_key = source.date_key
WHEN MATCHED THEN UPDATE ...
```

Druga próba, ten sam prompt:

```
MERGE fact_table AS target
USING staging_table AS source
ON target.product_id = source.product_id
   AND target.location_id = source.location_id
   AND target.date_key = source.date_key
WHEN MATCHED THEN UPDATE ...
```

Oba się kompilują. Oba wydają się sensowne. Żaden nie musi być poprawny.

Kluczowy wniosek

Nigdy nie zjrzało do twojej bazy danych.

Ani razu — dopóki nie daliśmy mu narzędzia, które mogło to zrobić.

Pewnie, lecz wiarygodnie błędne

Halucynacja to sposób działania generacji, a nie rzadki błąd.

- Wymyśla kolumny, które nie istnieją — z pełnym przekonaniem
- Proponuje relacje, które brzmią rozsądnie, ale nie ma ich w schemacie
- Cytuje ograniczenia platformy, które są całkowicie zmyślane
- Generuje komunikaty błędów, które wyglądają oficjalnie, ale nigdy nie istniały

Kluczowy wniosek

„Jeśli uruchomiło się bez błędu, to znaczy, że jest poprawne”
to najgroźniejsze zdanie w tej hurtowni.

Uzgodnione liczby są dowodem, nie kod wyjścia 0.

W naszej aplikacji

Każde ćwiczenie od tego miejsca odpowiada na te same cztery pytania.

- Jakim kontekstem dysponuje agent o tej hurtowni danych teraz?
- Co ma wytworzyć? Zielony build? Skrypt wdrożeniowy? Zapytanie uzgadniające?
- Jakich narzędzi może użyć? Tylko odczyt schematu? Dostęp do zapisu? Zewnętrzne API?
- Jak zweryfikujemy? Co znaczy „poprawne” i jak to mierzymy?

Deklaratywnie, nie imperatywnie

Nigdy nie piszesz ALTER

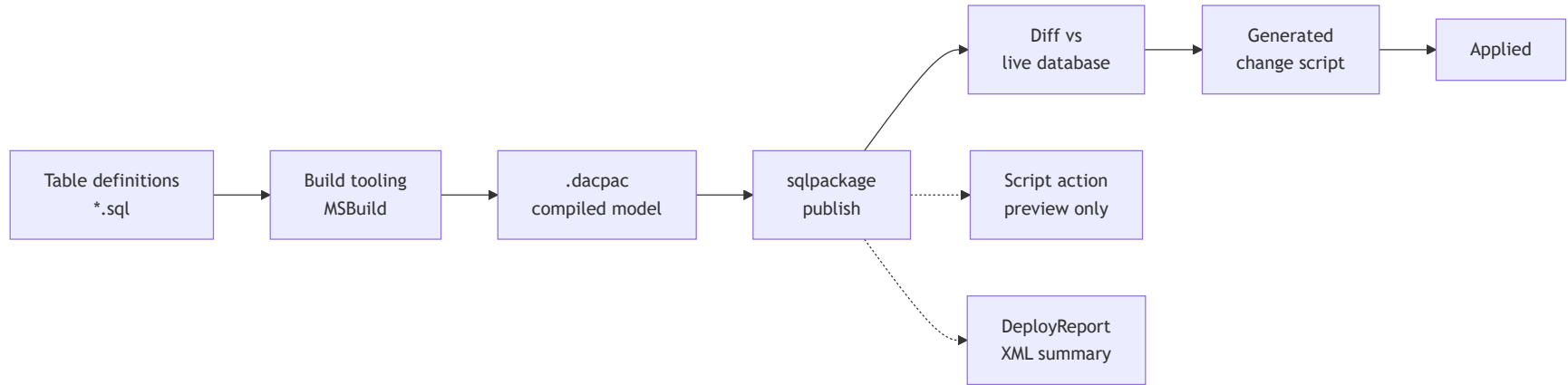
Problem

W narzędziu opartym na migracjach to ty autorujesz instrukcję `ALTER` .

Co się dzieje, gdy zamiast ciebie proponuje ją agent AI — dziewięć razy, na dziewięć różnych sposobów?

- Każda próba się kompiluje
- Każda wprowadza *jakiś* rodzaj zmiany
- Która z nich jest naprawdę poprawna?
- Jak przejrzyć skumulowany efekt?

Model



Kluczowy wniosek

Diff jest artefaktem pochodnym.

Jeśli go nigdy nie wygenerujesz, nigdy nie przejrzysz swojej zmiany.

Build kontra Publish

Build weryfikuje:

- Referencje do obiektów rozwiązywane są poprawnie
- Składnia T-SQL jest poprawna
- Zależności są w odpowiedniej kolejności
- Reguły analizy kodu przechodzą

Build nie może zweryfikować:

- Czy aktywna baza danych odpowiada modelowi?
- Czy zapytanie zwraca poprawne wyniki?
- Czy indeks zostanie użyty?
- Czy dane spełniają nowy constraint?

W naszej aplikacji

Proces, który właśnie zweryfikujemy:

- **Build** → tworzy skompilowany model (`.dacpac`)
- **Publish** → porównuje model z aktywną bazą danych w lokalnym kontenerze
- **Smoke test** → potwierdza, że kilka zapytań zwraca oczekiwane wyniki

Jeśli wszystkie trzy się powiedą, mamy **zweryfikowany stan bazowy** — mierzalny stan „przed”.

Nawyk powracający przez cały warsztat

Dostęp tylko do odczytu najpierw. Za każdym razem.

Problem

Agent z uprawnieniami do edycji bazy, na której nie zdobył jeszcze zaufania, to ryzyko, nie funkcja.

- Może proponować zmiany, zanim zrozumie bieżący stan
- Może wykonywać polecenia bez bramek weryfikacyjnych
- Może iterować na uszkodzonym baseline, nie zauważając tego
- Może modyfikować obiekty, od których zależą inne procesy

Co znaczy „potwierdzone”

Dostęp tylko do odczytu da się zweryfikować, a nie tylko założyć.

- Agent może otworzyć i streścić plik projektu bazy danych
- Agent może odczytać definicje tabel, widoków i procedur
- Agent wykonał zero poleceń deploymentowych lub DDL
- Agent zapisał zero plików w projekcie

To konkretna, sprawdzalna definicja — nie „vibe”.

Kluczowy wniosek

Weryfikacja to krok, nie sytuacja awaryjna.

Ta sama dyscyplina co przy każdej zmianie ETL:
dev → test → reconcile → deploy

W naszej aplikacji

Lista kontrolna weryfikacji dla dostępu tylko do odczytu:

1. Najpierw zaufaj workspace — niezaufany workspace po cichu wyłącza dostosowania
2. Poproś agenta o streszczenie pliku projektu i kilku kluczowych obiektów
3. Sprawdź panel diagnostics — potwierdź, że nie wywołano narzędzi do edycji ani wdrażania
4. Potwierdź w historii czatu — tylko operacje odczytu, bez zapisów

Ta sama lista kontrolna powtarza się przez cały warsztat — to wzorzec, nie jednorazowa konfiguracja.

Analiza repozytorium

Używaj AI do bezpiecznego eksplorowania nieznanego systemu

Cztery szczeble, każdy dodaje
ryzyko

"AI" to nie jest jedna rzecz

Problem

Wywołanie modelu, okno czatu, asystent z twoimi plikami i agent z narzędziami zachowują się zupełnie inaczej wobec tego samego pytania.

- Ten sam model
- Ten sam prompt
- Inne możliwości
- Inne tryby awarii

Cztery szczeble

Model	Token wejściowy → token wyjściowy. Brak pamięci, brak kontekstu poza bieżącym wejściem.
Chat	Generuje odpowiedzi na podstawie historii rozmowy. Nadal bez dostępu do stanu zewnętrznego.
Asystent	Trwałe instrukcje, dołączone pliki, niektóre narzędzia. Potrafi pobierać dane, ale nie planuje pracy wieloetapowej.
Agent	Planuje i wykonuje wiele działań w kierunku celu. Może się pomylić, sprawdzić i poprawić.

Agent to czat, który może się pomylić na głos — a
potem to sprawdzić

W tym warsztacie

- **Moduł 00:** weryfikacja tylko do odczytu
Tryb agenta, ale z ograniczeniami
- **Ten moduł:** eksploracja na zimno
Agent czyta repozytorium, żeby zmapować zależności
- **Późniejsze moduły:** generowanie planów
Agent proponuje zmiany i pokazuje swoje rozumowanie

Dlaczego tryb agenta ma tu znaczenie

Pierwszy prompt zimnego agenta działa tylko dlatego, że agent w **trybie agenta** może sam otworzyć procedurę ETL.

Okno czatu z wklejonym pytaniem nie mogłoby tego zrobić.

Najważniejszy wniosek

- Cztery szczeble: Model → Chat → Asystent → Agent
- Każdy dodaje możliwości — i ryzyko
- Agenci mogą **sprawdzać własną pracę**, używając narzędzi
- Eksploracja repozytorium wymaga trybu agenta z dostępem do plików

Narzędzia zamieniają opinie w weryfikowalne fakty

Zgadywanie vs. grepowanie

Problem

Bez narzędzi model dopełnia wzorzec, który widział **tysiące** razy.

Nie „wyszukuje” tabeli wymiarów — **przewiduje**, jak prawdopodobnie wygląda.

Przed narzędziami

Zapytanie: "Map the ETL pipeline for fact sales data"

Odpowiedź:

```
-- Likely structure (training-data prior):  
CREATE TABLE dbo.FactTable (  
    SaleID INT,  
    ProductKey INT,  
    CustomerKey INT,  
    DateKey INT,  
    Amount DECIMAL(18,2)  
);
```

- ✓ Płynnie
- ✗ Nie twój schemat

Po narzędziach

Zapytanie: "Map the ETL pipeline #codebase "

Odpowiedź:

```
Found pipeline:  
    stg.Transactions (staging)  
    → temp table (transformation)  
    → MERGE into fact table  
    → temporal history  
    → ETL log
```

Source: /etl/main-procedure.sql:47-89

- ✓ Cytuje prawdziwe pliki
- ✓ Zawiera numery linii

Nigdy nie zajrzał do twojej bazy danych. Ani razu.

(dopóki nie dasz mu narzędzi)

Wzorzec



1. Pytaj z `#codebase` lub odwołaniem do pliku
2. Agent otwiera plik — sprawdź sekcję **References**
3. Agent cytuje zawartość pliku wraz z numerami linii
4. Ty weryfikujesz, że cytat zgadza się z prawdziwym repozytorium

W VS Code

- Użyj trybu agenta Plan
- Dodaj `#codebase` do promptu, żeby zaznaczyć dostęp do repozytorium
- Sprawdź sekcję References — pokazuje pliki, które agent naprawdę przeczytał
- Jeśli nie pojawia się sekcja References, odpowiedź jest przewidywaniem, a nie wyszukiwaniem

⚠️ Częsty błąd: ufanie płynnej odpowiedzi bez sprawdzenia References

Najważniejszy wniosek

- Bez narzędzi model przewiduje na podstawie danych treningowych
- Z narzędziami agent czyta twoje rzeczywiste repozytorium
- Zawsze sprawdzaj sekcję **References**
- Ścieżka pliku + numer linii = weryfikowalny fakt

Prawdziwy zimny przebieg

Dostęp tylko do odczytu, bez pliku instrukcji

Co zrobił dobrze

- Odtworzył pełen pipeline ETL: staging → transformacja → MERGE → tabela faktów → historia
- Znalazł dokładne punkty edycji wewnątrz głównej procedury ładowania
- Zadał naprawdę dobre pytania doprecyzowujące:
 - typ danych i nulłowałość nowej kolumny
 - dostępność systemu źródłowego
 - wpływ grain na tabelę faktów
 - strategia backfillu dla danych historycznych

✓ Nie projektuj tego modułu wokół założenia, że AI jest głupie — nie było.

Teraz znajdź błąd

(jest na następnym slajdzie)

Zmyślona teza

"Zaktualizuj skrypt pre-deployment, aby wyłączyć śledzenie historii temporalnej podczas zmiany schematu... Publikowanie kończy się błędem platformy, gdy próbuje się dodać kolumnę do temporal table, podczas gdy historia nadal jest aktywna."

Co jest nie tak?

Zarówno reguła, jak i błąd są zmyślane

30-sekundowy test to obala

Dlaczego to ma znaczenie

-  Pewne siebie
-  Konkretnie
-  Cytuje wiarygodnie brzmiące ograniczenie platformy
-  Całkowicie zmyśłone

Realne ryzyko

Pójście za tą radą wyłączyłoby śledzenie historii temporalnej na produkcji — otwierając okno, w którym historia może zostać **bezpowrotnie utracona**, jeśli coś pójdzie nie tak podczas zmiany schematu.

Co jeszcze pominął

- Tabela logów ETL: cicho uznana za poza zakresem
- Zapytanie walidacyjne: pominięty check odrzuconych wierszy
- Mapowanie backfillu: pominięta synchronizacja danych po wdrożeniu
- Liczba plików: od razu padło „12 plików do zmodyfikowania”, zanim pojawiły się otwarte pytania

Wzorzec

Wszystkie pięć pominięć ma jedną wspólną przyczynę:

Agent nie miał **pliku kontekstu** opisującego:

- Co jest w zakresie
- Jakich wzorców walidacji się spodziewać
- Jak wygląda proces po wdrożeniu

Jak to wyłapać

- Sprawdź References: czy agent zacytował prawdziwy plik dla danej tezy?
- Przetestuj reguły platformy: jeśli cytuje komunikat błędu, odtwórz go
- Poproś o źródła: „pokaż mi plik, w którym ta restrykcja jest udokumentowana”
- Skontruj dokumentację: przeszukaj oficjalne dokumenty pod kątem cytowanego kodu błędu

Zimny przebieg to dowód, nie dogmat.

Najważniejszy wniosek

- Zimny przebieg był w większości poprawny — a mimo to podał jedną pewną, zmyśloną tezę
- Płynność ≠ dokładność
- Zawsze weryfikuj ograniczenia platformy na podstawie rzeczywistych testów lub oficjalnej dokumentacji
- Naprawa: daj agentowi kontekst o tym, co jest w zakresie i jakich wzorców ma się spodziewać

⚠️ Następny moduł: napisz plik instrukcji, który zapobiega tym pięciu pominięciom

Inżynieria kontekstu

Jedna strona markdown potrafi zrobić to, czego lepsze promptowanie nie potrafiło

Na jakiej krzywej jesteś?

Stagnacja kontra kumulacja

Koszt przepisywania

W każdej nowej sesji te same poprawki

- "Nie ruszaj wierszy historii — tabele temporalne tego pilnują"
- "Zawsze kwalifikuj odwołania do wymiarów schematem"
- "MERGE potrzebuje wszystkich trzech klauzul dla kolumn audytowych"
- "Sprawdź skrypt po wdrożeniu pod kątem logiki backfillu"

Inżynieria promptów

Jedno zadanie naraz

- Poprawka żyje w tej rozmowie
- Agent uczy się na to zadanie
- Następna sesja: czysta karta
- Umiejętność się wypląszcza

Inżynieria kontekstu

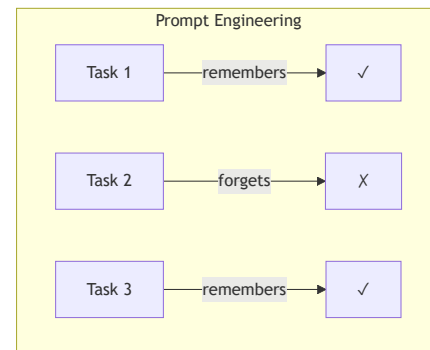
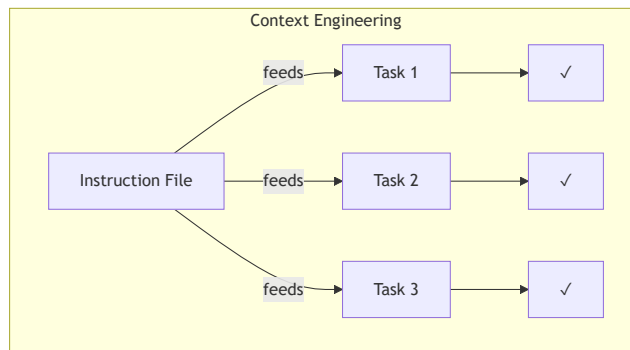
Każde zadanie, każda sesja

- Reguła żyje w kontroli wersji
- Agent ładuje ją automatycznie
- Każda sesja zaczyna z wiedzą
- Wiedza się kumuluje

Umiejętność promptowania ma sufit
Kontekst nie

Wzorzec

Raz zapamiętany czy za każdym razem?



W hurtowni danych

Większość błędów to *lokalny fakt*, nie rozumowanie

- Który schemat jest właścicielem tabel stagingowych?
- Czy kolumna grain jest dopuszczalna jako null?
- Czy można bezpośrednio ruszać wiersze historii?
- Skąd pochodzą kolumny audytowe?

Pięć nazwanych, zweryfikowanych błędów

Nie hipotetycznie — prawdziwy zimny run

Lista poprawek

Co musi zablokować plik instrukcji

Błąd	Co się zepsuło
Zmyślony ogranicznik platformy	Z przekonaniem stwierdził, że tabele temporalne nie pozwalają dodawać kolumn
Cicho zmieniony współdzielony artefakt	Zmodyfikował współdzieloną tabelę śledzenia ładowania bez pytania
Pominięty wymagany wynik	Pomiął wynik zapytania oczekiwany przez wywołującego
Pominięty skrypt backfillu	Zapomniał sprawdzić staged DDL w hookach po wdrożeniu
Struktura przed treścią	Zapytał „ile plików?” przed „o co prosi zgłoszenie?”

Błąd #1: Wymyślona reguła

Pewnie podany zakaz, razem z komunikatem o błędzie — oba zmyślane

- Twierdzenie: "Tabele temporalne nie pozwalają dodawać kolumn, gdy `SYSTEM_VERSIONING` jest ON"
- Podany skutek: Publikacja zakończy się błędem o konkretnym, wiarygodnym brzmieniu
- Rzeczywistość: SQL Server nie ma takiego ograniczenia — kolumny dodają się normalnie
- Sygnał ostrzegawczy: Zarówno reguła, jak i komunikat o błędzie zostały wymyślane na poczekaniu

Plik instrukcji to lista poprawek
nie lista życzeń

Co powinno trafić do pliku

Pięć kategorii, które łapią pięć błędów

- Platforma / wersja / dialekt — co naprawdę jest prawdą o tabelach temporalnych w SQL Server
- Zasady własności schematu — kto jest właścicielem stagingu, kto faktów, kto artefaktów współdzielonych
- Grain i semantyka temporalna — co oznacza dopuszczalność wartości null i brak null w projekcie
- Zabronione operacje — nigdy nie ruszaj historii bezpośrednio, nigdy nie rób backfillu bez skryptu
- Plan przed implementacją — najpierw otwarte pytania, potem struktura pliku

Zasada anty-rozrostu

Czy chciałbym to mieć w oknie kontekstu zadania niezwiązanego z tematem?

- Jeśli tak — to jest instrukcja (reguły temporalnych tabel dotyczą każdej zmiany)
- Jeśli nie — to jest dokumentacja (zrzut schematu na 500 linii to szum, nie sygnał)
- Test: czy ta reguła pomogłaby przy zadaniu dotyczącego zupełnie innej tabeli?
- Trzymaj plik w granicach — 50–100 linii w zupełności wystarczy

Ten sam agent. Ten sam
prompt.

Jeden plik różni wszystko.

Ustawienie testu A/B

Metoda

Zamrożony prompt, uruchomiony na zimno dwa razy

- Prompt: "Przeanalizuj wpływ dodania kolumny audytowej snapshot do potoku ładowania tabeli faktów"
- Wariant A: Bez pliku instrukcji — agent startuje na zimno
- Wariant B: Plik instrukcji w katalogu głównym repo — agent łąduje go automatycznie
- Oba uruchomienia: Nowe sesje tego samego dnia, ten sam model
- Ocena według: Pięciu skatalogowanych błędów z bazowego uruchomienia

Wynik

5 z 5 naprawione

Błąd	Bez pliku	Z plikiem
Zmyślony ogranicznik	Stwierdził, że temporal zabrania kolumn	Zacytowano prawdziwy dokument referencyjny
Cicha zmiana artefaktu	Samowolnie zmodyfikował współdzieloną tabelę	Zgłoszono to jako otwarte pytanie
Pominięte zapytanie	Pomiął wynik oczekiwany przez wywołującego	Wypisano jako jawny punkt zmiany
Pominięty backfill	Zapomniat o kontroli po wdrożeniu	Oznaczono jako działanie CRITICAL
Struktura przed treścią	Najpierw zapytał „ile plików?”	Struktura z otwartymi pytaniami jako pierwsza

5 z 5

Jedna strona markdown zrobiła to, czego lepsze promptowanie nie potrafiło

Czego to nie dowodzi

Ramowanie uczciwości

- Nie dowodzi: że działa to dla każdego modelu, każdego zadania albo znacznie większego repozytorium
- Nie dowodzi: 100% eliminacji wszystkich możliwych przyszłych błędów
- Co dowodzi: w tym zadaniu, w tym kodzie, plik naprawił wszystkie pięć skatalogowanych błędów
- Prawdziwy test: kiedy uruchomisz ten sam eksperyment na własnym pliku, jeszcze dziś

Teraz ty to uruchamiasz

Twoja kolej: zmierz n z 5

- Napisz własny plik instrukcji (50–100 linii, struktura pięciu kategorii)
- Zamroź prompt z wcześniejszego zadania
- Uruchom go na zimno dwa razy: raz bez pliku, raz z plikiem
- Oceń: ile z pięciu błędów naprawił twój plik?
- **To jest pomiar** — trzymaj się tabeli punktacji, a nie „wydaje mi się, że jest lepiej”

Mała zmiana SQL, pełny cykl

Jedno niejednoznaczne zgłoszenie, od początku do końca

Prompt jako specyfikacja

Prompt to zgłoszenie robocze, nie pytanie

Dwa prompty

Lewy: Developer A

```
make the view better
```

Prawy: Developer B

```
Add VAT-exclusive net amount column  
to daily sales aggregate view for  
Finance's monthly VAT reconciliation.
```

Którego byś przydzielił?

- Jedno jest pytaniem
- Drugie jest specyfikacją
- Tylko jedno da się wykonać

Zapytaj salę: Które przekazałbyś juniorowi?

CRISPE Framework

Lista kontrolna kompletnych promptów

Context Środowisko, istniejący stan systemu

Role Kim agent ma być

Instruction Konkretny wymagany krok

Specification Kryteria akceptacji, przypadki brzegowe

Performance Mierniki sukcesu, standardy jakości

Examples Przypadki referencyjne, oczekiwane wzorce

CRISPE jako wykrywacz luk

Przeczytaj sześć liter.
Której brakuje w *twoim* promptcie?

To jest właściwa instrukcja do ćwiczenia, które zaraz nastąpi.

W naszej aplikacji hurtowni danych

Mapowanie prawdziwego zgłoszenia na CRISPE

Jest

- Context ✓
- Role ✓
- Instruction ✓

Brakuje

- **Specification** — Co dokładnie znaczy "net"? Jak zmienia się VAT? Co z NULL-ami?
- **Performance** — Precyzja? Zaokrąglenia? Co oznacza "poprawne"?
- **Examples** — Dostarczone przypadki referencyjne w danych startowych

Następny krok: Uzupelnij luki, zanim napiszesz jakikolwiek kod.

Zgłoszenie

Jedno zdanie. Pięć niewypowiedzianych założeń.

Czego zgłoszenie nie mówi

Cztery krytyczne założenia pozostawione implicite

- Co znaczy "net" — przed VAT, przed rabatem, czy jedno i drugie?
- Oczekiwania wobec precyzji i zaokrągleń
- Czy stawka podatku może się różnić wewnątrz grupy sprzedażowej
- Dokładna nazwa kolumny i wymagany typ danych

Zapytaj salę: Podaj piąte niewypowiedziane założenie.

Fakty, które musisz przeczytać

Nie zgadywać

```
-- Rabat jest już uwzględniony  
GrossAmount =  
    (Qty * UnitPrice) - DiscountAmount
```

Pułapka: Założenie, że "Gross" oznacza przed rabatem

```
-- Stawka VAT przenoszona per wiersz  
SELECT  
    p.VatRate,  
    f.GrossAmount  
FROM FactSales f  
JOIN DimProduct p ...
```

Kluczowa myśl: Stawka może różnić się w zależności od wiersza, nawet w tej samej grupie agregacji

Przed / Po

Luka w specyfikacji

Jednozdaniowe zgłoszenie

```
Add net amount column to daily  
sales view for VAT reconciliation.
```

Brakujący kontekst: jakie formuły istnieją, jakie stawki obowiązują

4-punktowe kryteria akceptacji

1. `NetAmount = GrossAmount / (1 + VatRate)`
per wiersz, przed agregacją
2. Precyzja `DECIMAL(18,4)`, bez zaokrąglania do wyświetlania
3. Obsługa zmienne stawki wewnątrz grupy
4. Propaguj NULL, gdy wejście ma wartość NULL

Teraz junior mógłby to wykonać.

Kluczowa myśl

Jeśli człowiek nie mógłby na tym działać,
model też nie może.

Ta sama dyscyplina specyfikacji obowiązuje niezależnie od tego, kto wykonuje pracę.

Wiersz z przypadkiem brzegowym

Jeden wiersz. Jedna niestandardowa stawka. Jeden błąd.

Dwie kandydackie formuły

Obie się kompilują. Obie działają. Tylko jedna jest poprawna.

```
-- Wzór A: Zakodowana na sztywno stawka standardowa  
SUM(GrossAmount) / 1.23  
  
-- Wzór B: Zastosowana stawka na poziomie wiersza  
SUM(GrossAmount / (1 + VatRate))
```

Która obsługuje zmienne stawki?

Na danych ze standardową stawką

- Obie zwracają identyczne wyniki
- Różnicy nie da się wykryć
- Testy przechodzą w obu przypadkach

Przy niestandardowej stawce

- Wzór A: zakłada 23% wszędzie
- Wzór B: respektuje rzeczywistą stawkę dla każdego wiersza
- Luka staje się widoczna

Prześledź jeden wiersz

Śledzenie obliczenia produktu o specjalnej stawce

Wiersz brzegowy

- Kategoria produktu ze specjalną stawką
- Zastosowana niestandardowa stawka podatku
- Inna niż standardowe 23%
- Kwota brutto z już uwzględnionym rabatem

```
-- Wzór A (założenie zakodowane na sztywno)
```

```
GrossAmount / 1.23
```

```
→ zakłada standardową stawkę
```

```
-- Wzór B (świadomy wiersz)
```

```
GrossAmount / (1 + VatRate)
```

```
→ używa rzeczywistej stawki z wymiaru
```

Wynik: Mierzalna różnica liczbowa między tymi dwoma podejściami

Luka pomiarowa

Zakodowana na sztywno stawka standardowa: jeden wynik

Rzeczywista stawka z wiersza: inny wynik

Różnica: znaczna i mierzalna

To rzeczywista, zmierzona rozbieżność z produkcyjnie podobnego scenariusza.

Kluczowa myśl

Moje dane testowe decydowały
które defekty dało się w ogóle wykryć.

Wiersz o specjalnej stawce był już obecny w danych startowych, zanim powstał ten ticket.

W praktyce

Jak defekt ujawnia się w weryfikacji

- Zestaw testów referencyjnych sprawdza agregaty względem znanych poprawnych obliczeń
- Jeden wiersz o specjalnej stawce izolowany: porównanie sum przed i po agregacji
- Formuła ze stałą stawką: test nie przechodzi na tym konkretnym wkładzie wiersza
- Formuła świadoma wiersza: test przechodzi dla wszystkich wariantów stawki

Wniosek: Weryfikacja jest obowiązkowa, nie opcjonalna — a różnorodność danych testowych decyduje o tym, co w ogóle możesz wykryć.

Analiza wpływu

Jedna kolumna, dziewięć obiektów, jeden plan przed jedną linią kodu

src: ./impact-matrix-methodology.pl.md

src: ./review-colleagues-analysis.pl.md

src: ./the-reveal.pl.md

Implementacja wielu obiektów

Systematyczny przegląd diffu i kontrola zakresu

Dziewięć obiektów, jedna kolumna

Ścieżka propagacji od staging do historii

Tabela propagacji

Każdy punkt styku dla nowej kolumny przepływającej przez hurtownię

Etap potoku	Typ obiektu	Cicha porażka?
Tabela stagingowa	Definicja kolumny	✗ Kompilacja
DDL tabeli tymczasowej	Definicja kolumny	✓ Brak kolumny
Ranked CTE	Projekcja kolumny	✓ Brak wartości
MERGE USING	Odwołanie do kolumny	✓ Błędna wartość
MERGE INSERT	Lista kolumn + VALUES	✓ W połowie zaimplementowane
MERGE UPDATE	Klauzula SET	✓ W połowie zaimplementowane
Tabela historii	Definicja kolumny	✗ Wdrożenie

Policz ciche przypadki

Większość punktów propagacji zawodzi po cichu

- Głośne awarie: 2 przy kompilacji, 1 przy wdrożeniu
- Ciche awarie: pozostała większość
- Ryzyko: odhaczanie „kolumna dodana” po zobaczeniu jej tylko raz

Dwa głośne wyjątki

Awarie, które i tak byś wychwycił

Brakująca kolumna w stagingu

```
INSERT INTO #TempTable  
  (Amount, Region) -- ← brak nowej kolumny  
SELECT Amount, Region, TrackingValue  
FROM staging.Source
```

Błąd: Niezgodność liczby kolumn

Kiedy: Pierwsza próba INSERT

Widoczność: Natychmiastowa awaria

Niezgodność schematu historii

```
-- Tabela faktów ma nową kolumnę  
ALTER TABLE dbo.FactTable  
  ADD TrackingColumn INT NULL;  
  
-- Definicja tabeli historii bez zmian  
-- (wciąż ma starą listę kolumn)
```

Błąd: Walidacja narzędzia wdrożeniowego

Kiedy: Próba wdrożenia

Widoczność: Kontrola przed wdrożeniem

W aplikacji referencyjnej

Diff, który dziś przeglądamy

```
CREATE TABLE #TransformedData (  
    SalesKey INT,  
+   TrackingColumn INT NULL,  
    Amount DECIMAL(18,2)  
);  
  
;WITH Ranked AS (  
    SELECT  
        SalesKey,  
        Amount,  
+   TrackingColumn,  
        ROW_NUMBER() OVER ( ... ) AS Rnk  
    FROM staging.SalesSource  
)  
  
INSERT INTO #TransformedData  
    (SalesKey, Amount, TrackingColumn)  
SELECT SalesKey, Amount, TrackingColumn  
FROM Ranked WHERE Rnk = 1;
```

Pytanie do przeglądu: Czy to pełna propagacja?

Lista kontrolna kontra rzeczywistość



Obecne w diffie

- DDL tabeli tymczasowej
- Projekcja CTE
- Lista kolumn INSERT
- Lista VALUES INSERT



Znaki zapytania

- Gałąź `MERGE` INSERT?
- Gałąź `MERGE` UPDATE?
- Synchronizacja tabeli historii?
- Metadane dziennika ładowania?

Lista kontrolna propagacji zmusza do szukania braku, nie tylko obecności.

Gałąź, której nikt nie czyta

MERGE ma dwie ścieżki kodu. Recenzenci czytają jedną.

Gałąź INSERT

Nowe wiersze dostają pełne traktowanie

```
MERGE dbo.FactTable AS tgt
USING #TransformedData AS src
  ON tgt.SalesKey = src.SalesKey
WHEN NOT MATCHED BY TARGET THEN
  INSERT (SalesKey, Amount, TrackingColumn)
VALUES (src.SalesKey, src.Amount, src.TrackingColumn);
```

-  Kolumna obecna na liście INSERT
-  Wartość obecna na liście VALUES
-  Pochodzi ze stageru przez tabelę tymczasową
- Uważny recenzent zatrzymuje się tutaj

Gałąź UPDATE

Istniejące wiersze są niewidoczne

```
MERGE dbo.FactTable AS tgt
USING #TransformedData AS src
  ON tgt.SalesKey = src.SalesKey
-- ...
WHEN MATCHED THEN
  UPDATE SET
    tgt.Amount = src.Amount,
    tgt.Region = src.Region;
-- ← brakuje przypisania TrackingColumn
```

-  Brak przypisania dla nowej kolumny
- Inne kolumny aktualizują się poprawnie
-  Defekt polega na braku, nie na obecności

Dlaczego to jest niewidoczne

Nie da się zobaczyć czegoś, czego nie ma

✓ Pierwsze ładowanie: wszystkie wiersze trafiają przez INSERT → nowa kolumna jest wypełniona

✓ Kompilacja się udaje, testy przechodzą, status zwraca 'SUCCEEDED'

✗ Drugie ładowanie: istniejące wiersze są aktualizowane → nowa kolumna zostaje przy starej wartości

Kluczowa intuicja

"Kolumna JEST tam —
w połowie MERGE
która nie jest błędem."

Zapytanie wykrywające

Weryfikacja behawioralna, nie czytanie kodu

```
-- Uruchom ładowanie dwa razy z wartością, o której wiadomo, że się zmienia
EXEC etl.LoadFactTable @TrackingValue = 1;
EXEC etl.LoadFactTable @TrackingValue = 2;

-- Sprawdź, czy MERGE rzeczywiście zaktualizował wiersze
SELECT
    RowsInserted,
    RowsUpdated,
    RowsDeleted
FROM dbo.EtlLog
WHERE ProcedureName = 'etl.LoadFactTable'
    AND TrackingValue = 2;

-- Asercja: RowsUpdated > 0 (nie tylko Status='SUCCEEDED')
```

- Czytanie kodu: łatwo przegapić brak
- Test behawioralny: dowodzi, że gałąź UPDATE się wykonała i dotknęła danych
- Asercja: nie zero zaktualizowanych, konkretne klucze faktycznie się zmieniły

Pół-MERGE to wzorzec

Nie jednorazowy błąd

Typowe kształty

- Nowa kolumna w INSERT, brak w UPDATE
- Nowa kolumna w UPDATE, brak w INSERT
- Detekcja zmian uwzględnia stare kolumny, pomija nowe
- Komentarz mówi „wszystkie kolumny”, ale kod nie

Dlaczego to się dzieje

- Listy INSERT i UPDATE utrzymywane oddzielnie
- Kopiuj-wklej z istniejącego wzorca
- „Skompilowało się” brzmi jak sukces
- Ścieżka UPDATE nie jest testowana aż do drugiego ładowania

Czego diff nie powinien zawierać

Zatwierdzona lista obiektów to także lista wszystkiego, co jest poza zakresem

Prawdziwy diff scope creep

Zmiany poza zakresem w produkcyjnym pull requeście

Zmodyfikowane pliki (9):

+ etl/procedures/LoadFactTable.sql	← zatwierdzone
+ etl/tables/FactTable.sql	← zatwierdzone
+ etl/tables/FactTableHistory.sql	← zatwierdzone
+ reporting/views/WeeklySummary.sql	← niezatwierdzone
+ scripts/maintenance/ArchiveOldData.sql	← niezatwierdzone
+ deploy/ApplyIndexTuning.sql	← niezatwierdzone

- Zatwierdzone: 3 kluczowe obiekty ETL z analizy wpływu
- Scope creep: 3 dodatki typu „przy okazji”
- Problem: uwaga recenzenta jest teraz podzielona 50/50

Uwaga recenzenta jest ograniczona

W niedawnym teście duża liczba dodanych linii

ukryła niewielki, krytyczny defekt

Scope creep to nie hojność — to szum

Kuzyn nieodporny na NULL

Ten sam plik, ta sama linia, jedno wywołanie funkcji dalej

✗ Wygląda dobrze, zawodzi po cichu

```
WHEN MATCHED
  AND (tgt.Amount <> src.Amount
       OR tgt.TrackingColumn <> src.TrackingColumn)
THEN UPDATE SET ...
```

Problem: `NULL <> NULL` to `UNKNOWN`, nigdy `TRUE`

Skutek: Wiersze z `NULL` nigdy się nie aktualizują

✓ Porównanie odporne na NULL

```
WHEN MATCHED
  AND (tgt.Amount <> src.Amount
       OR ISNULL(tgt.TrackingColumn, -1)
          <> ISNULL(src.TrackingColumn, -1))
THEN UPDATE SET ...
```

Dlaczego: `ISNULL` zamienia `NULL` na porównywalny sentinel

Skutek: Wiersze z `NULL` aktualizują się poprawnie

Komentarz kontra kod

Ufaj, ale weryfikuj

```
-- Detekcja zmian obejmuje wszystkie kolumny (odporne na NULL)
WHEN MATCHED
  AND (tgt.Amount <> src.Amount
       OR tgt.TrackingColumn <> src.TrackingColumn)
    -- ↑ to nie jest odporne na NULL
THEN UPDATE SET ...
```

- Komentarz twierdzi: „odporne na NULL”
- Kod jest: nieodporny na NULL
- Wniosek: traktuj każdy komentarz jak hipotezę do obalenia
- Test: uruchom z wartościami NULL i sprawdź, że aktualizacje > 0

W aplikacji referencyjnej

Dzisiejsza rubryka oceny

Wymagana obecność

Z checklisty propagacji:

-  Kolumna w tabeli stagingowej
-  DDL tabeli tymczasowej
-  Projekcja CTE
-  Odwołanie `MERGE USING`
-  Kolumna + wartość INSERT
-  Przypisanie w UPDATE SET
-  Synchronizacja tabeli historii
-  Metadane dziennika ładowania

Wymagana nieobecność

Z zatwierdzonej listy obiektów:

-  Widoki raportowe
-  Skrypty konserwacyjne
-  Strojenie indeksów
-  Narzędzia kopii zapasowej
-  Zmiany wyłącznie formatowania
-  Zmienione nazwy plików
-  Jakikolwiek obiekt spoza planu

Zatwierdzony plan określa ZARÓWNO to, co ma być obecne, JAK i to, czego nie może być.

Przegląd diffu to kontrola z dwóch stron

Wszystko z checklisty jest obecne

✗ Nic spoza zatwierdzonego zakresu nie jest obecne

Jedno brakujące: niepełna implementacja

Jedno nadmiarowe: scope creep

Część 2

Migracja, testowanie, przegląd i samodzielne dostarczenie

Moduł 06

Migracja temporalna

Zmiana modelu i migracja danych to różne operacje z różnymi regułami

Zmiana modelu vs. migracja danych

Kolumna już istnieje. Istniejąca historia jej nie ma.

Problem

- Dodano nową kolumnę śledzącą: przeszła przez staging, transformację i samą tabelę faktów
- Najnowsze wiersze: kolumna ma prawdziwą wartość
- Istniejące wcześniej wiersze: kolumna ma `NULL` — zarówno w tabeli bieżącej, jak i w historii
- Pytanie: to problem schematu czy problem danych?

Trzy niezmienniki

Tylko silnik zapisuje historię

- Bezpośredni `INSERT / UPDATE / DELETE` na tabeli historii jest blokowany
- Wiersze historii powstają automatycznie, gdy wiersz bieżący jest aktualizowany albo usuwany

Okres = UTC od początku transakcji

- `ValidFrom` i `ValidTo` rejestrują czas transakcji, nie czas aplikacji
- Brak luk, brak nakładania się okresów

Okresy są domknięte-otwarte [`ValidFrom`, `ValidTo`)

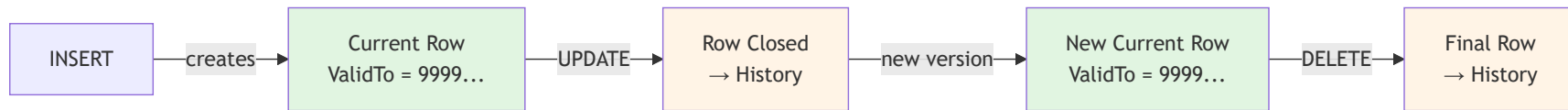
- Wiersz kwalifikuje się od `ValidFrom` (włącznie) do `ValidTo` (wyłącznie)
- Wiersze bieżące: `ValidTo = '9999-12-31 23:59:59.9999999'`
- Wiersze zamknięte: `ValidTo` ustawiane, gdy powstaje następna wersja

Kluczowa myśl

Dokładnie jedna wersja klucza kwalifikuje się w danej chwili

To właśnie gwarantuje `[ValidFrom, ValidTo)`.

Cykl życia wiersza



- Zielone: tabela bieżąca — dokładnie jedna wersja na klucz
- Pomarańczowe: tabela historii — zero lub więcej zamkniętych wersji na klucz
- Każde przejście stanu to zamknięta transakcja — kolumny okresu zapisują ten moment

W naszej aplikacji

Stan początkowy

- Tabela faktów: mieszanka wartości `NULL` sprzed migracji i nowszych wartości nie- `NULL`
- Tabela historii: wszystkie wiersze powstały przed nową kolumną — każdy pojedynczy ma `NULL`
- Cel: nadać każdemu wcześniejszemu wierszowi prawdziwą, uczciwą wartość

Szkielet bezpiecznej migracji

Wsteczne uzupełnianie historii jest zablokowane wprost — o ile znasz maszynę stanów

Oczywiste podejście zawodzi

Próba: bezpośredni `UPDATE` na tabeli historii

```
UPDATE HistoryTable  
SET TrackingColumn = 'sentinel-value'  
WHERE TrackingColumn IS NULL;
```

Wynik: `Msg 13561: Cannot update rows in a temporal history table`

Kluczowa myśl: przynajmniej ta porażka jest głośna. Następna pułapka będzie cicha.

Przed / po

Zablokowana ścieżka

```
-- Direct UPDATE: fails
UPDATE HistoryTable
SET Column = Value
WHERE Column IS NULL;
```

✗ Silnik blokuje operację natychmiast

Bezpieczna ścieżka

```
-- 1. Check current state
IF ... 'HISTORY_TABLE'
-- 2. Disable versioning
ALTER TABLE ... SET (SYSTEM_VERSIONING = OFF);
-- 3. Backfill both tables
UPDATE CurrentTable ... WHERE ... IS NULL;
UPDATE HistoryTable ... WHERE ... IS NULL;
-- 4. Re-enable with check
ALTER TABLE ... SET (
    SYSTEM_VERSIONING = ON (
        ... DATA_CONSISTENCY_CHECK = ON
    )
);
```

Kluczowa myśl

`SYSTEM_VERSIONING = OFF` nie jest obejściem

To jedyny sposób na bezpośredni zapis do historii — a ponowne włączenie nie jest opcjonalnym sprzątniem

Wzorzec migracji

```
-- Step 1: State check (idempotent guard)
IF EXISTS (
    SELECT 1 FROM sys.tables t
    WHERE temporal_type = 2 -- HISTORY_TABLE
)
BEGIN
    -- Step 2: Disable versioning (guarded)
    ALTER TABLE CurrentTable SET (SYSTEM_VERSIONING = OFF);

    -- Step 3: Backfill both tables via dynamic SQL
    EXEC sp_executesql N'UPDATE CurrentTable SET Col = @val WHERE Col IS NULL',
        N'@val INT', @val = 99;
    EXEC sp_executesql N'UPDATE HistoryTable SET Col = @val WHERE Col IS NULL',
        N'@val INT', @val = 99;

    -- Step 4: Re-enable with consistency check
    ALTER TABLE CurrentTable SET (
        SYSTEM_VERSIONING = ON (
            HISTORY_TABLE = HistoryTable,
            DATA_CONSISTENCY_CHECK = ON
        )
    );

    -- Step 5: Post-condition asserts
```

W praktyce

Decyzja o sentinelu

- Wybierz wartość, która znaczy „załadowane, zanim istniało śledzenie”
- Opisz to w komentarzu, nie w nazwie zmiennej
- Przykład: 99 albo -1 albo 0 — cokolwiek Twoja domena rozpoznaje jako „to nie jest prawdziwe ID”
- To jest decyzja biznesowa, nie techniczny domyślny wybór

Zanim napiszesz to samodzielnie

Dwie pułapki odkryte podczas budowania tego wzorca

Obie prawdziwe. Obie zweryfikowane. Jedna z nich jest cicha.

Co prawie poszło nie tak

Dwie nazwane pułapki, obie prawdziwe, obie odkryte podczas budowania tego ćwiczenia

Pułapka 1 — Uzupełnisz złą tabelę

Pułapka: aktualizujesz tabelę bieżącą, gdy versioning nadal jest włączony

```
-- Looks reasonable, runs without error
UPDATE CurrentTable
SET TrackingColumn = 'sentinel-value'
WHERE TrackingColumn IS NULL;
```

- ☒ Instrukcja kończy się powodzeniem
- ☒ Brak błędu, brak ostrzeżenia
- ☒ Tabela bieżąca ma teraz wartość sentinela w każdym wierszu
- Tabela historii bez zmian — nadal same `NULL`

Pułapka 1 — szkoda, zmierzona

Liczba wierszy historii

- Przed: wartość bazowa
- Po: prawie podwojona
- Nowe wiersze: śmieciowe wersje utworzone przez `UPDATE` backfillu

Wartości w historii

- Cel: wypełnić `NULL` -e sentinelem
- Wynik: nadal same `NULL` w istniejących wcześniej wierszach historii
- Backfill zaktualizował wiersze bieżące, nie historię
- SQL Server utworzył nowe wersje historii — dokładnie tak, jak zaprojektowano

Pułapka 2 — Separatory wsadów pod dołączaniem skryptu

Pułapka: separatory `GO` zachowują się inaczej w zależności od tego, jak skrypt jest uruchamiany

```
ALTER TABLE CurrentTable SET (SYSTEM_VERSIONING = OFF);  
GO -- ← Works in sqlcmd, fails under deployment tools  
UPDATE CurrentTable SET Col = 'value' WHERE Col IS NULL;
```

-  Działa w interaktywnym `sqlcmd`
-  Nie działa w narzędziach wdrożeniowych — nawet na pustej tabeli
- Błąd: `Msg 13561` — ten sam błąd co przy bezpośredniej aktualizacji historii
- Przyczyna: niektóre mechanizmy dołączania skryptów po cichu łączą instrukcje przez granice `GO`

Pułapka 2 — naprawa

Opakuj każdą zależną instrukcję w `sp_executesql`

```
-- Disable versioning (no wrapper needed for DDL)
ALTER TABLE CurrentTable SET (SYSTEM_VERSIONING = OFF);

-- Backfill via dynamic SQL — always compiled as its own fresh batch
EXEC sp_executesql N'UPDATE CurrentTable SET Col = @val WHERE Col IS NULL',
                  N'@val INT', @val = 99;
EXEC sp_executesql N'UPDATE HistoryTable SET Col = @val WHERE Col IS NULL',
                  N'@val INT', @val = 99;

-- Re-enable versioning (no wrapper needed for DDL)
ALTER TABLE CurrentTable SET (
    SYSTEM_VERSIONING = ON (HISTORY_TABLE = HistoryTable, DATA_CONSISTENCY_CHECK = ON)
);
```

- Dynamiczny ciąg SQL jest zawsze kompilowany jako świeży wsad
- Działa identycznie w `sqlcmd`, narzędziach wdrożeniowych i SSMS
- Ta sama technika, której używają własne dokumenty Microsoftu dla czyszczenia retencji

Wzorzec

Ta sama przyczyna źródłowa w obu przypadkach

Zakładanie, że narzędzie zachowa się tak jak sesja interaktywna, którą w nim testowałeś

Najważniejsze wnioski

Liczba wierszy historii to Twój test

- Policz przed wyłączeniem versioning
- Policz po ponownym włączeniu
- Asseruj, że liczby są identyczne
- Jeśli nie są: ślad audytu został uszkodzony

Wdrożenie ≠ sesja interaktywna

- Separator `G0` : zależne od narzędzia
- Dynamiczny SQL: respektowany zawsze
- Testuj migrację pod narzędziem wdrożeniowym, nie tylko w `sqlcmd`
- „Uruchomiło się bez błędu” niczego nie dowodzi

Testy regresji

Testuj kod generowany przez AI — nie tylko go czytaj

Testuj kod AI, nie tylko go czytaj

W aplikacji webowej błąd AI powoduje awarię. W T-SQL przechodzi cicho, ale z błędnymi danymi.

Problem

- Prawie nic nie jest sprawdzane typami na granicach obiektów T-SQL
- `NULL` propaguje się przez wyrażenia zamiast rzucać wyjątek
- Instrukcje `MERGE` z radością dopasowują zero wierszy bez ostrzeżenia
- Procedura może zwrócić sukces, a jednocześnie uszkodzić każdy wiersz

Oś głośności

Kluczowa obserwacja

Ryzyko defektu AI jest odwrotnie proporcjonalne do tego, jak głośno się ogłasza.

Ciche defekty wmykają się do produkcji.

Wzorzec w
praktyce

W tym module

Zasialiśmy cztery defekty w potoku ładowania tabeli faktów.

- Każdy dotyka tej samej nowej funkcji: kolumny śledzącej, która musi przejść przez staging → transformację → tabelę końcową
- Każdy ma brzmiące wiarygodnie uzasadnienie, które mógłby wygenerować agent
- Każdy kończy się bez wyjątku
- Pytanie: gdzie lądują na osi głośności?

Cztery terminale, wszystkie zielone

Ten sam smoke test, cztery subtelnie uszkodzone warianty ETL

Przed: Co sprawdza naiwny smoke test

```
-- Obiekt istnieje
IF OBJECT_ID('etl.LoadFactTable') IS NULL
    RAISERROR('Procedure missing', 16, 1);

-- Procedura uruchamia się bez wyjątku
EXEC etl.LoadFactTable @TrackingValue = 100;

-- Status oznacza sukces, brak odrzuceń
SELECT Status, RowsRejected
FROM dbo.EtlLog
WHERE ProcedureName = 'etl.LoadFactTable';
```

Po: Czego nie sprawdza

- Wartości na ścieżce aktualizacji — czy ponowne załadowanie naprawdę aktualizuje istniejące wiersze?
- Zgodność typów między tabelami — czy kolumny staging i target pasują do siebie?
- Unikanie sentinela — czy brakujące wartości stają się wiarygodnie wyglądającymi placeholderami?
- Kolejność deduplikacji — która wersja wygrywa, gdy klucze się zderzają?

Ujawnienie

Cztery zasiane defekty, cztery różne kształty:

Archetyp	Co się psuje	Jak to jest wykrywane
Porównanie bez ochrony	Ścieżka aktualizacji po cichu pomija istniejące wiersze	Sprawdź, że dla znanego klucza zmieniła się rzeczywista wartość
Niezgodność typów	Typ kolumny w tabeli historii nie zgadza się z tabelą faktów	Zapytanie do metadanych wykrywa to w czasie testu; narzędzie wdrażające wykryłoby to przy publikacji
NULL→sentinel	Brakujące wartości stają się <code>0</code> albo <code>-1</code>	Przeskanuj kolumnę o wartości biznesowej w poszukiwaniu sentinelów
Stara-wersja-wygrywa	Deduplikacja zachowuje starsze dane zamiast nowszych	Sprawdź konkretną wartość biznesową po znanym zdarzeniu

Kluczowa obserwacja

Jeden defekt został wykryty przez narzędzie wdrożeniowe — zanim uruchomiono jakikolwiek test.

Pozostałe trzy zwróciły sukces bez żadnych błędów.

Wzorzec: asercje
wartości
biznesowych

W praktyce: przypadek starego wiersza, który wygrywa

```
-- Załaduj 8 wierszy z TrackingValue = 100, Quantity = 2.0
-- Następnie załaduj ponownie te same 8 kluczy z TrackingValue = 101, Quantity = 999.0

EXEC etl.LoadFactTable @TrackingValue = 101;

SELECT Status, RowsUpdated FROM dbo.EtlLog; -- SUCCEEDED, RowsUpdated = 8
SELECT ProductKey, Quantity FROM dbo.FactTable
WHERE ProductKey IN (1,2,3,4,5,6,7,8);
-- Uszkodzone: Quantity = 2.0 (stare). Poprawne: Quantity = 999.0.
```

- RowsUpdated jest dodatnie w obu przypadkach — liczy dopasowanie MERGE, a nie zmianę wartości
- Status to 'SUCCEEDED' w obu przypadkach — nie został rzucony wyjątek
- Tylko wartość biznesowa ujawnia uszkodzenie

Teatr weryfikacji

```
-- ☒ Obiekt istnieje
-- ☒ Uruchamia się bez wyjątku
-- ☒ Status = 'SUCCEEDED'
-- ☒ RowsRejected = 0

-- Lista kontrolna zakończona!
```

Potwierdza: Procedura nie uległa awarii.

Myślenie mutacyjne

```
-- Którego znanego złego builda
-- ten test ma obalić?

-- Jeśli odpowiedź brzmi „żadnego”,
-- usuń test –
-- to teatr weryfikacji.
```

Standard: Wartość testu mierzy się tym, na czym może się on wyłożyć, a nie tym, co przechodzi.

Złudzenie

Status = 'SUCCEEDED' — co to naprawdę oznacza

Problem

```
SELECT Status, RowsInserted, RowsUpdated, RowsRejected  
FROM dbo.EtlLog  
WHERE ProcedureName = 'etl.LoadFactTable'  
AND Status = 'SUCCEEDED';
```

- Co to znaczy: Procedura zakończyła się bez rzucenia wyjątku
- Czego to nie znaczy: Dane są poprawne
- Co z tego wnioskujemy: Jeśli jest zielono, to zadziało
- Luka: Zero informacji o poprawności

Kluczowa obserwacja

Zielony nic nie znaczy.

Oznacza tylko, że nie został rzucony wyjątek.

Test Green-by-Construction

Test napisany na podstawie implementacji.

```
-- Jeśli kod robi X,  
-- a test sprawdza X,  
-- test przechodzi na mutancie  
-- który też robi X  
-- (ale nie powinien).
```

Przechodzi na: Błędnej implementacji, z której został wyprowadzony.

Test świadomy mutacji

Test napisany na podstawie reguły biznesowej.

```
-- Reguła biznesowa mówi Y.  
-- Test sprawdza Y.  
-- Jeśli kod robi zamiast tego X,  
-- test zawodzi –  
-- nawet jeśli X „wygląda sensownie”.
```

Zawodzi na: Każdej implementacji, która narusza regułę.

Standard

Wartość testu mierzy się tym, na czym może się wyłożyć, a nie tym, co przechodzi.

- Czy zawodzi, gdy ścieżka aktualizacji pomija istniejące wiersze?
- Czy zawodzi, gdy deduplikacja zachowuje złą wersję?
- Czy zawodzi, gdy brakująca wartość staje się wiarygodnie wyglądającym placeholderem?
- Jeśli odpowiedź na wszystkie trzy brzmi „nie”, test to teatr weryfikacji.

Most do wydajności

W następnym module każde pomiarowe wskazanie też będzie wyglądało pewnie przy pierwszym odczycie.

- Stoper pokazuje 1.2s bez przedziałów błędu
- Plan zapytania pokazuje Actual Rows = 1000 bez ostrzeżenia
- Agent cytuje oficjalne wytyczne z brzmiącym wiarygodnie uzasadnieniem
- Ta sama dyscyplina: nie ufaj pierwszemu pewnemu sygnałowi.

Diagnostyka wydajności

Mierz przed diagnozą, przed naprawą

src: ./measure-first.pl.md

src: ./three-attempts.pl.md

src: ./extend-dont-add.pl.md

Automatyczny przegląd + przygotowanie commita/MR

Zamień kryteria przeglądu w powtarzalny proces

Skill, a nie plik promptu

Każdy przegląd do tej pory był świeżym promptem. Dziś staje się plikiem.

Problem

Rozproszone kryteria, wpisywane od nowa za każdym razem

- Lista kontrolna przeglądu żyje w notatkach lub pamięci
- Każde nowe zadanie: formułujesz prompt od zera
- Kryteria dryfują między przeglądami
- Brak spójnego formatu między zmianami

Przed / Po

Plik promptu

`.prompt.md` — tylko Copilot / VS Code

- Leży w `.github/prompts/`
- Ładuje się tylko w sesjach VS Code
- Wykluczony z sesji Agent Host
- Nieprzenośny między narzędziami

Plik skilla

`.github/skills/{name}/SKILL.md`

- Ładuje się na żądanie w wielu narzędziach
- Działa w Claude Code, Copilot i Agent Host
- Może dołączać pliki pomocnicze
- Przenośny między środowiskami

„Przenośne” jest prawdziwe dla instrukcji i skilli
Fałszywe dla plików promptu

Wzorzec

Ścieżki między narzędziami

Narzędzie	Ścieżka
-----------	---------

Claude Code	<code>.claude/skills/{name}/SKILL.md</code>
----------------	---

Copilot	<code>.github/skills/{name}/SKILL.md</code>
---------	---

Nazwa katalogu = nazwa skilla

W naszej aplikacji

Struktura skilla przeglądu hurtowni:

```
.github/skills/sql-change-review/  
├─ SKILL.md           # Główna procedura  
├─ checklist.md       # Osiem kategorii przeglądu  
├─ reconciliation.sql  # Szablon zapytania walidacyjnego  
└─ examples/  
    ├─ good-diff.diff  # Przypadek testowy bez findings  
    └─ bad-diff.diff   # Przypadek testowy z wykrytym błędem
```

Co tak właściwie oznacza „przejrzane”?

Scalanie doraźnych kryteriów w jedną stałą listę

Problem

Przegląd, który za każdym razem zmienia kształt, nie jest procesem

- Niejasne prośby dają niejasne wyniki
- Inny recenzent = inne skupienie
- Brak sposobu audytu tego, co sprawdzono
- Pewnie brzmiące twierdzenia wymykają się bez kontroli

Przed / Po

Niejasny prompt

Please review this SQL change

- Brak struktury
- Brak odpowiedzialności
- Ustalenia zależą od nastroju modelu
- Nie da się zmierzyć fałszywych pozytywów

Stała rubryka

Osiem nazwanych kategorii:

1. **Zakres** — czy to robi to, o co proszono?
2. **Zależności** — kolejność migracji bezpieczna?
3. **Bezpieczeństwo migracji** — idempotentna, odwracalna?
4. **NULL / wykrywanie zmian** — porównania poprawne?
5. **Testy** — automatyczny regres jest obecny?
6. **Bezpieczeństwo** — ryzyko iniekcji/dostępu?
7. **Wydajność** — strategia indeksów ma sens?
8. **Rejestracja projektu** — nowe pliki w buildzie?

Każda kategoria musi zostać obsłużona findingiem
lub wyraźnym „brak problemu”
Nigdy nie pomijana po cichu

Wzorzec

Wymagany blok wyjściowy

```
**Finding**: [what's wrong]  
**Risk**: Critical | High | Medium | L  
**Evidence**: [diff line # or docs cit  
**Recommendation**: [how to fix]
```



Kończy się checklistą przed
scaleniem

W naszej aplikacji

Twarde zasady przeglądu hurtowni:

- Nigdy nie podawaj zachowania temporal table jako faktu bez cytatu
- Nigdy nie rozwiązuj pytania o zakres po cichu — wypisz je
- Potwierdź, że każdy nowy plik migracji ma odpowiadający wpis w pliku projektu
- Jeśli nie da się zweryfikować twierdzenia wyłącznie z diffa, oznacz je jako unverified

Czy AI review naprawdę działa?

To nie jest pytanie retoryczne — mierzymy to

Problem

Komenda przeglądu musi być jednocześnie precyzyjna i kompletna

- Zbyt agresywna → fałszywe pozytywy podkopują zaufanie
- Zbyt cicha → przegapia prawdziwy błąd
- Trzeba testować oba przypadki: czysty kod i wadliwy kod
- Jeden przebieg to nie jest dowód

Przed / Po

Przebieg 1: Czysty diff

Poprawne ulepszenie temporal table

- 0 ustaleń na poziomie Medium+
- Wszystkie kategorie obsłużone
- Werdykt: *"Gotowe do scalenia z jednym warunkowym zatwierdzeniem"*
- Brak fałszywych pozytywów

Przebieg 2: Wadliwy diff

Zawiera porównanie niebezpieczne dla NULL

- 1 Critical finding
- Kategoria: NULL / wykrywanie zmian
- Dokładna lokalizacja zacytowana
- Wykryto podłożony defekt

„Komentarz bezpośrednio powyżej stwierdza...”

„...a kod implementuje”

„dokładnie to niebezpieczne porównanie, przed
którym ostrzega”

Wzorzec

Sprzeczność komentarza i kodu

Gdy ostrzeżenie i implementacja
sobie przeczą

```
-- Handles NULL safely by treating  
-- NULL = NULL as a match  
WHERE tgt.Column <> src.Column
```

Silniejszy sygnał niż każda z części
osobno

W naszej aplikacji

To jest komenda, której użyjesz na swoim własnym diffie w capstone

- Bez przygotowanych diffów testowych
- Bez siatki bezpieczeństwa od trenera
- Twoja zmiana, twoja rubryka, twoje evidence
- Dyscyplina wobec fałszywych pozytywów, którą dziś budujesz, ochroni cię wtedy

Dowód AI okazał się błędem

Przegląd nie kończy się na diffie — kończy się też na MR

Problem

Wygenerowane zapytanie walidacyjne wyglądało poprawnie, uruchomiło się bez błędu i zwróciło wiersz

- Sekcja walidacji po wdrożeniu
- Zapytanie potwierdzające, że procedura ładowania zadziałała
- Uruchomiło się czysto, bez komunikatu o błędzie
- Zwróciło wiersz z danymi

Przed / Po

Uszkodzone (zwracało wiersz)

```
DECLARE @LoadId INT;  
EXEC etl.LoadProcedure  
    @LoadId = @@IDENTITY;  
  
SELECT @LoadId; -- Returns a value!
```

- Używa @@IDENTITY nieprawidłowo
- Brakuje słowa kluczowego OUTPUT
- @LoadId pozostaje NULL
- Zapytanie „sukcesem” waliduje nic

Poprawione (naprawdę waliduje)

```
DECLARE @LoadId INT;  
EXEC etl.LoadProcedure  
    @LoadId = @LoadId OUTPUT;  
  
SELECT @LoadId; -- Now captures result
```

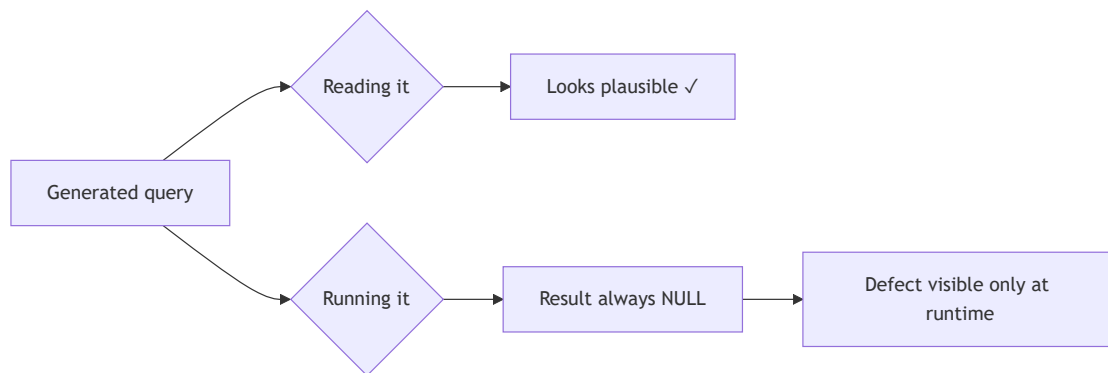
- Dodaje słowo kluczowe OUTPUT
- Parametr przechwytuje wartość zwracaną
- Wynik to faktycznie identyfikator ładowania
- Weryfikacja jest prawdziwa

Prawdopodobne, ale bierne

Legalna składnia, brak błędu, zwraca wiersz

i niczego nie waliduje

Wzorzec



Czytanie ≠ Uruchamianie

W naszej aplikacji

Nowa zasada dla każdego procesu generowania MR:

- Każde polecenie w sekcji walidacji musi zostać **wykonane** na prawdziwej bazie danych przed dodaniem
- W przeciwnym razie oznacz je jako *"sugerowane, niezweryfikowane"*
- Wygenerowana proza nie jest bardziej godna zaufania niż wygenerowany kod
- Zastosuj tę samą dyscyplinę sprawdzania faktów do dokumentacji

Ćwiczenie końcowe

Samodzielna zmiana end-to-end

Czym jest ta sesja

- Czas na pracę, nie czas na nauczanie
- Pełny przepływ pracy: eksploracja → plan → implementacja → testy → review → dokumentacja
- Nie ma listy plików — o zakresie decyduje Twoja analiza wpływu
- Przy każdym rozwidleniu decyzję podejmujesz samodzielnie

Rozwiedlenie decyzji

- NOT NULL czy nullable? Inne ograniczenia, inna walidacja
- Odrzucić czy przyjąć? Kolumny obowiązkowe wymagają ostrzejszych zabezpieczeń ETL
- Indeks teraz czy później? Uzasadniaj wymaganiami wprost, nie odruchem
- Ekspozycja raportowa? Możliwość filtrowania wpływa na warstwę zapytań

Omówienie po pracy

Zastanów się nad swoimi decyzjami

- Co zasugerował Ci język ticketu?
- Które decyzje podjąłeś świadomie, a które odziedziczyłeś z poprzedniej pracy?
- Gdzie własny review wychwycił lukę?
- Co zmieniłbyś, gdybyś uruchamiał ten przepływ pracy jeszcze raz jutro?

Pytania?

i odpowiedzi